

QA Bug Report

Web Application Quality Assurance Assessment — Task 1

App Under Test: Conduit — RealWorld Demo (demo.realworld.show)
Test Date: May 21, 2026
Tester: QA Engineer
App Type: Full-stack social blogging platform (Medium clone)
Testing Method: Manual exploratory testing across all major user flows

Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Duplicate Sign-up Allowed with Same Email	1. Register with email@test.com 2. Log out 3. Register again with the same email@test.com	Expected: Error — 'Email already in use' Actual: New account is created silently, allowing duplicate accounts	Critical	Backend missing unique-email constraint check on registration endpoint, or validation not enforced server-side
2	Invalid Email Format Accepted on Sign-up	1. Go to Sign Up 2. Enter 'notanemail' in the email field 3. Click Sign Up	Expected: Inline validation error — 'Enter a valid email address' Actual: Account is created with malformed email	High	Missing client-side regex validation and no server-side email format enforcement on the /users endpoint
3	No Error When Changing Password to Same Value	1. Log in 2. Go to Settings 3. Enter the current password in the password field 4. Click Update	Expected: Warning — 'New password must be different from current password' Actual: Request succeeds silently with no feedback	Medium	Settings update endpoint does not compare hashed new password with existing one before accepting the change
4	No Password Strength Indicator	1. Go to Sign Up 2. Enter a weak password like '12345678' 3. Observe UI feedback	Expected: Password strength meter (weak/medium/strong) shown Actual: Only minimum-length (8 chars) is enforced; '12345678' is accepted with no warning	Low	No password strength library integrated in frontend; only basic minlength HTML attribute used
5	Vague 'Credentials Invalid' Error on Login Failure	1. Go to Sign In 2. Enter a registered email with the wrong password 3. Click Sign In	Expected: Specific error such as 'Incorrect password' or at least a user-friendly message Actual: Displays generic 'email or password' is invalid — acceptable but provides no action guidance	Low	API returns a single generic error object; frontend does not map error codes to user-friendly messages

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
6	Article Form Submits Without Inline Field-Level Validation Hints	1. Log in 2. Click 'New Article' 3. Leave title, description, and body blank 4. Click Publish	Expected: Inline red highlights under each empty required field Actual: A top-level error list appears but individual fields are not highlighted — user cannot immediately see which field failed	Medium	Validation errors returned as a generic list from the API; frontend does not map error keys back to individual form fields
7	Mobile Layout — Content Overflows to Screen Edges (No Horizontal Padding)	1. Open the app on a mobile device or Chrome DevTools device emulation (375px width) 2. Browse the home feed or an article	Expected: Content has left/right padding so text is not flush against screen edges Actual: Text and cards touch the screen boundaries; readability is poor on small screens	Medium	CSS container lacks responsive padding rules (e.g., padding: 0 16px) at small breakpoints; likely a missing media query

Root Cause Analysis

Selected Bug: Bug #1 — Duplicate Sign-up Allowed with Same Email

Symptom

When a user attempts to create a new account using an email address that is already registered in the system, the application creates a second account instead of rejecting the request. This means two distinct accounts can share the same email, which breaks a fundamental identity constraint.

What is happening

The registration flow sends a POST request to the `/api/users` endpoint with the new user's credentials. The server processes and persists the user record without first querying the database to check whether the email already exists. As a result, duplicate rows with the same email column value are inserted into the users table.

Why it happens

The root cause is missing server-side uniqueness enforcement. In a correctly configured database, the email column should carry a `UNIQUE` constraint, which would cause the `INSERT` to throw an error on a duplicate. Additionally, the API layer should explicitly validate for existing emails before attempting the insert, returning a `422 Unprocessable Entity` response with an appropriate error message. Both of these safeguards appear to be absent.

Client-side gap

There is also no client-side pre-check (e.g., a debounced lookup as the user types their email) that could give immediate feedback. While a server-side check is non-negotiable for security, a client-side check would improve UX.

How to fix it

1. Database: Add a `UNIQUE` constraint on the email column — e.g., `ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email)`.
2. API layer: Before `INSERT`, run `SELECT 1 FROM users WHERE email = $1` and if a row exists, return `HTTP 422` with `{ errors: { email: ['has already been taken'] } }`.
3. Frontend: Display the returned error object inline beneath the email field so the user knows exactly what to correct.

Summary

Severity	Count	Examples
Critical	1	Duplicate email sign-up
High	1	Invalid email format accepted
Medium	3	No same-password error, article form UX, mobile padding
Low	2	No password strength meter, vague login error
Total	7	

The most critical issue is the duplicate email sign-up vulnerability, which must be fixed before any production deployment as it undermines user identity integrity. The high-severity invalid email acceptance is equally important. Medium and low severity issues primarily affect user experience and should be addressed in subsequent iterations.